

BASH SCRIPTING FUNDAMENTALS

Variables & Quoting

Store data safely and avoid word-splitting bugs

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026



Agenda

- 1 Section 1: Variable Fundamentals — 7 topics
- 2 Section 2: Quoting Essentials — 7 topics
- 3 Section 3: Word Splitting and Globbing — 7 topics
- 4 Section 4: Defaults and Required Values — 7 topics
- 5 Section 5: String Parameter Expansion — 7 topics
- 6 Section 6: Declarations and Variable Lifetime — 7 topics
- 7 Section 7: Environment and PATH — 7 topics
- 8 Section 8: Positional Parameters and Options — 7 topics

- 9 Section 9: Reading Input and IFS — 7 topics
- 1 Section 10: Arrays and Arithmetic — 7 topics
- 0
- 1 Section 11: Substitution and Special Variables — 7 topics
- 1
- 1 Section 12: Formatting, Comparisons, and Review — 7 topics
- 2

1 Section 1

Variable Fundamentals

1.1 Variable Assignment

- A Bash variable stores a text value under a name.

Assign a word

```
course=bash
```

- Assignment uses name=value with no spaces around the equal sign.

Assign a path

```
log_file=app.log
```

1.2 No Spaces Around Equals

- Spaces around the equal sign change the meaning of an assignment.

Correct assignment

```
name=Sam
```

- Bash treats words separated by spaces as command names and arguments.

Incorrect shape

```
name = Sam
```

1.3 Reading Variables

- A dollar sign reads the value stored in a variable.

Read a value

```
name=Sam  
echo "$name"
```

- The variable name itself is not printed when you use dollar expansion.

Use in text

```
course=bash  
echo "Course: $course"
```

1.4 Variable Names

- Bash variable names commonly use letters, numbers, and underscores.

Use a clear name

```
report_date=2026-07-28
```

- A variable name cannot start with a number.

Use underscore

```
input_file=data.txt
```

1.5 Empty Variables

- A variable can be set to an empty value.

Assign empty value

```
name=""  
printf "[%s]\n" "$name"
```

- Empty and unset are different states, but both may print as nothing.

Unset then print

```
unset name  
printf "[%s]\n" "$name"
```

1.6 unset - Remove a Variable

- The unset command removes a variable from the current shell.

Unset a variable

```
token=abc  
unset token
```

- After unset, normal expansion produces an empty value unless strict checks are used.

Show after unset

```
unset token  
printf "[%s]\n" "$token"
```

1.7 readonly - Protect a Variable

- The readonly command prevents a variable from being changed or unset.

Create readonly value

```
readonly app_name="demo"
```

- It is useful for constants that should stay fixed during a script.

Declare readonly

```
declare -r region="us-east"
```

2 Section 2

Quoting Essentials

2.1 Double Quotes

- Double quotes keep a value together as one word after expansion.

Preserve spaces

```
name="Ada Lovelace"  
echo "$name"
```

- Variables and command substitutions still expand inside double quotes.

Expand in sentence

```
item=file  
echo "Selected: $item"
```

2.2 Single Quotes

- Single quotes preserve text exactly as written.

Print literal dollar

```
echo '$HOME'
```

- Variables do not expand inside single quotes.

Print literal text

```
echo 'Use $name here'
```

2.3 Unquoted Variables

- An unquoted variable can be split into multiple words.

Unsafe display

```
name="Ada Lovelace"  
echo $name
```

- It can also trigger filename matching when the value contains wildcard characters.

Safer display

```
name="Ada Lovelace"  
echo "$name"
```

2.4 \$var Versus Quoted Var

- The form \$var expands the variable and then lets the shell process the result.

One argument safely

```
file="my notes.txt"  
ls "$file"
```

- The quoted form keeps the expanded value as one argument.

Multiple words unsafely

```
file="my notes.txt"  
ls $file
```

2.5 Braces Around Variable Names

- Braces mark exactly where a variable name begins and ends.

Add suffix safely

```
name=report  
echo "${name}.txt"
```

- They are helpful when text touches the variable name.

Avoid name confusion

```
day=Mon  
echo "${day}day"
```

2.6 Escaping Special Characters

- A backslash can remove special meaning from the next character.

Literal dollar

```
echo "Price is \$5"
```

- Escaping is useful inside double quotes when you need a literal dollar sign.

Literal quote

```
echo "She said \"hi\""
```

2.7 Nested Quoting

- Nested quoting appears when one command contains another command string.

Single outside

```
bash -c 'echo "$HOME"'
```

- Choose the outer quotes so the inner command expands at the intended time.

Double outside

```
name=Sam  
bash -c "echo hello"
```

3 Section 3

Word Splitting and Globbing

3.1 Word Splitting

- After unquoted expansion, Bash can split text into words.

Observe splitting

```
items="one two"  
printf "<%s>\n" $items
```

- The default split characters are spaces, tabs, and newlines.

Prevent splitting

```
items="one two"  
printf "<%s>\n" "$items"
```

3.2 Globbing After Expansion

- Globbing is filename matching with patterns such as star and question mark.

Unsafe pattern

```
pattern="*.txt"  
printf "%s\n" $pattern
```

- Unquoted variable values can become glob patterns after expansion.

Literal pattern

```
pattern="*.txt"  
printf "%s\n" "$pattern"
```

3.3 Splitting Before Globbing

- Bash performs word splitting before filename matching on unquoted results.

Show risky value

```
value="logs *.txt"  
printf "<%s>\n" $value
```

- A value with spaces and stars can turn into many separate arguments.

Keep one value

```
value="logs *.txt"  
printf "<%s>\n" "$value"
```

3.4 IFS Default Separators

- IFS is the variable Bash uses for some word splitting operations.

Show default effect

```
list="a b c"  
printf "%s\n" $list
```

- By default, it includes space, tab, and newline characters.

Set a comma separator

```
IFS=,  
line="a,b,c"  
printf "%s\n" $line
```

3.5 Custom IFS for One Read

- A temporary IFS assignment can change how read separates fields.

Split CSV fields

```
IFS=, read -r a b c <<< "red,green,blue"
```

- Putting IFS before read limits the assignment to that command.

Print fields

```
printf "%s %s %s\n" "$a" "$b" "$c"
```

3.6 Disable Globbing Briefly

- The set -f command disables filename expansion in the current shell.

Turn globbing off

```
set -f
```

- The set +f command turns filename expansion back on.

Turn globbing on

```
set +f
```

3.7 Safe Arguments Pattern

- Treat variable values as data unless you intentionally want shell syntax.

Safe file argument

```
file="my report.txt"  
cat "$file"
```

- Double-quote each variable when passing it as a command argument.

Safe directory argument

```
dir="old logs"  
mkdir -p "$dir"
```

4 Section 4

Defaults and Required Values

4.1 Default Value with Colon Dash

- The expansion `${var:-word}` uses `word` when `var` is unset or empty.

Use default name

```
name=${USER:-student}  
echo "$name"
```

- It does not assign the default back into the variable.

Default first argument

```
target=${1:-.}  
ls "$target"
```

4.2 Assign Default with Colon Equals

- The expansion `${var:=word}` assigns `word` when `var` is unset or empty.

Assign default port

```
: "${port:=8080}"
```

- After the expansion, the variable keeps the default value.

Show assigned value

```
echo "$port"
```

4.3 Require Value with Colon Question

- The expansion `${var:?message}` stops with an error when `var` is unset or empty.

Require a variable

```
: "${config:?config is required}"
```

- It is a compact way to require important inputs.

Require first argument

```
: "${1:?usage: script FILE}"
```

4.4 Alternate Value with Colon Plus

- The expansion `${var:+word}` uses `word` only when `var` is set and not empty.

Optional flag

```
debug=1  
flag=${debug:+- -verbose}
```

- It is useful for adding optional flags or labels.

Print alternate

```
name=Sam  
echo "${name:+present}"
```

4.5 Defaults for Script Arguments

- Positional parameters can use the same default expansion as named variables.

Default directory

```
dir=${1:-.}  
ls "$dir"
```

- The expression `${1:-.}` means use the first argument or the current directory.

Default name

```
name=${1:-student}  
echo "$name"
```

4.6 Empty Versus Unset in Defaults

- Colon forms treat empty and unset variables the same.

Colon treats empty as missing

```
x=""  
echo "${x:-fallback}"
```

- Forms without the colon only test whether the variable is unset.

No colon keeps empty

```
x=""  
echo "${x-fallback}"
```

4.7 Quote Default Expansions

- Default expansions can produce values with spaces.

Quoted default

```
name=${1:-Ada Lovelace}  
printf "%s\n" "$name"
```

- Quoting the whole expansion keeps the chosen value as one argument.

Quoted path default

```
dir=${1:-my files}  
ls "$dir"
```

5 Section 5

String Parameter Expansion

5.1 String Length

- The expansion `${#var}` returns the length of a variable value.

Measure a word

```
name=bash  
echo "${#name}"
```

- For simple ASCII text, the length is the number of characters.

Measure input

```
value="hello world"  
printf "%s\n" "${#value}"
```

5.2 Substring Expansion

- The expansion `${var:offset:length}` extracts part of a value.

First four characters

```
date=20260728  
echo "${date:0:4}"
```

- Offsets start at zero for the first character.

Month characters

```
date=20260728  
echo "${date:4:2}"
```

5.3 Replace First Match

- The expansion `${var/pattern/replacement}` replaces the first matching part.

Replace one dash

```
name=app-prod  
echo "${name/-/_}"
```

- The pattern uses shell pattern rules, not full regular expressions.

Replace extension

```
file=report.txt  
echo "${file/.txt/.csv}"
```

5.4 Replace All Matches

- The expansion `${var//pattern/replacement}` replaces every matching part.

Replace all dashes

```
name=app-prod-east  
echo "${name// - /_}"
```

- It is useful when a repeated separator needs to change.

Replace spaces

```
title="daily report"  
echo "${title// /_}"
```

5.5 Remove Shortest Suffix

- The expansion `${var%pattern}` removes the shortest matching suffix.

Remove extension

```
file=report.txt  
echo "${file%.txt}"
```

- It is often used to remove a file extension.

Remove after last dot

```
file=a.b.txt  
echo "${file%.*}"
```

5.6 Remove Longest Suffix

- The expansion `${var%%pattern}` removes the longest matching suffix.

Keep before first dot

```
file=a.b.txt  
echo "${file%%.*}"
```

- It is useful when a pattern can match more than one amount of text.

Remove path tail pattern

```
path=/tmp/app/log.txt  
echo "${path%%/*}"
```

5.7 Remove Prefix Patterns

- The expansion `${var#pattern}` removes the shortest matching prefix.

Remove shortest prefix

```
path=/tmp/app.log  
echo "${path#*/}"
```

- The expansion `${var##pattern}` removes the longest matching prefix.

Remove longest prefix

```
path=/tmp/app.log  
echo "${path##*/}"
```

6 Section 6

Declarations and Variable Lifetime

6.1 declare - Describe Variables

- The declare command creates variables and can assign attributes.

Declare a variable

```
declare name="Sam"
```

- It is a Bash builtin, so it is mainly for Bash scripts.

Show declared variables

```
declare -p name
```

6.2 declare -i - Integer Attribute

- The -i attribute asks Bash to treat assignments as arithmetic.

Declare integer

```
declare -i count=1
```

- This can be convenient for counters and simple numeric state.

Assign arithmetic

```
count=2+3  
echo "$count"
```

6.3 declare -r - Readonly Attribute

- The -r attribute creates a readonly variable.

Readonly with declare

```
declare -r app="demo"
```

- Readonly variables cannot be reassigned or unset in the same shell.

Inspect readonly value

```
declare -p app
```

6.4 declare -a - Indexed Array

- The -a attribute declares an indexed array.

Declare array

```
declare -a names
```

- Indexed arrays store multiple values under one variable name.

Assign array values

```
names=( "Ann" "Bo" )
```

6.5 declare -A - Associative Array

- The -A attribute declares an associative array.

Declare map

```
declare -A ports
```

- Associative arrays use string keys instead of only numeric indexes.

Assign by key

```
ports[web]=80
```

6.6 Shell Variables

- A shell variable exists inside the current shell process.

Create shell variable

```
mode=dev
```

- Child commands do not automatically receive ordinary shell variables.

Read in same shell

```
echo "$mode"
```

6.7 export - Send to Child Processes

- Exporting a variable places it in the environment for child processes.

Export a setting

```
export MODE=dev
```

- Programs started after export can read the exported value.

Child shell reads it

```
bash -c 'echo "$MODE"'
```

7 Section 7

Environment and PATH

7.1 Environment Versus Shell Variables

- Environment variables are inherited by child processes.

Shell only

```
color=blue  
bash -c 'echo "$color"'
```

- Shell variables stay in the current shell unless exported.

Exported value

```
export color=blue  
bash -c 'echo "$color"'
```

7.2 Inspect Environment Variables

- The printenv command shows values that are present in the environment.

Print one value

```
printenv PATH
```

- It does not show every ordinary shell variable.

Print all environment

```
printenv
```

7.3 Export While Assigning

- Bash can assign and export a variable in one command.

Assign and export

```
export APP_ENV=dev
```

- This is common for configuration values used by tools.

Use exported value

```
bash -c 'echo "$APP_ENV"'
```

7.4 Temporary Environment for One Command

- A variable assignment before a command can set environment only for that command.

Set for one command

```
APP_ENV=test env | grep APP_ENV
```

- The current shell variable is not permanently changed by that temporary assignment.

Set locale for date

```
LC_ALL=C date
```

7.5 Append to PATH Carefully

- Appending adds a directory to the end of PATH.

Append bin directory

```
PATH="$PATH:$HOME/bin"
```

- Commands in existing PATH directories keep priority over the appended directory.

Export after append

```
export PATH="$PATH:$HOME/bin"
```

7.6 Prepend to PATH Carefully

- Prepending adds a directory to the front of PATH.

Prepend local bin

```
PATH="$HOME/bin:$PATH"
```

- A prepended directory can override commands found later in PATH.

Check chosen command

```
command -v mytool
```

7.7 Remove Environment Values

- The unset command removes a variable from the shell and its future environment.

Unset exported value

```
export MODE=dev  
unset MODE
```

- Already-running child processes keep their own copies.

Confirm missing value

```
printenv MODE
```

8 Section 8

Positional Parameters and Options

8.1 \$0 and \$1

- \$0 usually contains the script name or the shell name.

Print script name

```
echo "script=$0"
```

- \$1 contains the first positional argument passed to a script or function.

Print first argument

```
echo "first=$1"
```

8.2 \$# - Argument Count

- \$# expands to the number of positional arguments.

Print count

```
echo "count=$#"
```

- It is useful for checking whether the user supplied enough input.

Require one argument

```
[ "$#" -ge 1 ] || exit 1
```

8.3 \$@ - All Arguments

- \$@ represents all positional arguments.

Forward arguments

```
printf "<%s>\n" "$@"
```

- When quoted as "\$@", each original argument stays separate.

Call another script

```
./worker.sh "$@"
```

8.4 \$* - Joined Arguments

- \$* also represents all positional arguments.

Join arguments

```
printf "<%s>\n" "$*
```

- When quoted as "\$*", the arguments join into one string.

Compare with at

```
printf "<%s>\n" "$@"
```

8.5 Quoted \$@

- Quoted "\$@" preserves empty arguments and arguments with spaces.

Loop safely

```
for arg in "$@"; do  
    echo "$arg"  
done
```

- Each argument remains its own word after expansion.

Forward safely

```
bash helper.sh "$@"
```

8.6 shift - Consume Arguments

- The shift command removes the first positional argument.

Shift once

```
echo "$1"  
shift  
echo "$1"
```

- After shift, the old second argument becomes the new first argument.

Shift in a loop

```
while [ "$#" -gt 0 ]; do  
    shift  
done
```

8.7 getopts Intro

- The getopts builtin parses short options such as -v and -f file.

Parse one flag

```
while getopts "v" opt; do
    echo "$opt"
done
```

- It updates variables so your script can react to each option.

Parse option value

```
while getopts "f:" opt; do
    echo "$OPTARG"
done
```

9 Section 9

Reading Input and IFS

9.1 read -r

- The read command stores input from standard input into variables.

Read one line

```
read -r line
```

- The -r option prevents backslashes from being treated as escapes.

Print the line

```
printf "%s\n" "$line"
```

9.2 read -p

- The -p option shows a prompt before reading input.

Prompt for name

```
read -r -p "Name: " name
```

- It is convenient for small interactive scripts.

Use the answer

```
echo "Hello, $name"
```

9.3 read Multiple Fields

- Read can assign separate input fields to multiple variables.

Read two fields

```
read -r first last
```

- IFS controls how the line is split into those fields.

Show fields

```
printf "%s,%s\n" "$first" "$last"
```

9.4 IFS with read

- Setting IFS before read changes the separator for that read command.

Read colon fields

```
IFS=: read -r user shell <<< "sam:/bin/bash"
```

- This is useful for colon-separated or comma-separated text.

Print parsed fields

```
printf "%s uses %s\n" "$user" "$shell"
```

9.5 Here-String

- A here-string sends a short string to a command's standard input.

Feed read

```
read -r word <<< "hello"
```

- The syntax uses three less-than characters before the string.

Count characters

```
wc -c <<< "hello"
```

9.6 while read Loop

- A while read loop processes input one line at a time.

Read a file

```
while read -r line; do  
    echo "$line"  
done < input.txt
```

- The read -r form preserves backslashes in each line.

Read command output

```
printf "a\nb\n" | while read -r x; do  
    echo "$x"  
done
```

9.7 Read Lines into an Array

- The readarray builtin reads lines into an indexed array.

Read file lines

```
readarray -t lines < input.txt
```

- The -t option removes trailing newlines from each element.

Print first line

```
printf "%s\n" "${lines[0]}"
```

10

Section 10

Arrays and Arithmetic

10.1 Indexed Arrays

- An indexed array stores multiple values under one variable name.

Create an array

```
names=( "Ann" "Bo" "Cy" )
```

- Indexes start at zero in Bash.

Read first item

```
echo "${names[0]}"
```

10.2 Array Expansion

- The expansion "\${array[@]}" produces each array element as a separate word.

Print each item

```
printf "<%s>\n" "${names[@]}"
```

- This preserves elements that contain spaces.

Pass to command

```
mkdir -p "${dirs[@]}"
```

10.3 Array Length

- The expansion `${#array[@]}` returns the number of array elements.

Count names

```
names=( "Ann" "Bo" )  
echo "${#names[@]}"
```

- It counts elements, not characters.

Check nonempty

```
[ "${#names[@]}" -gt 0 ] && echo "has names"
```

10.4 Associative Arrays

- An associative array uses text keys instead of numeric indexes.

Create a map

```
declare -A port=( [web]=80 [ssh]=22)
```

- Bash 4 and newer support associative arrays.

Read by key

```
echo "${port[web]}"
```

10.5 Arithmetic Expansion

- The expression `$(( ... ))` performs integer arithmetic.

Add numbers

```
echo "$((2 + 3))"
```

- Variables inside arithmetic expansion do not need a dollar sign.

Increment value

```
count=1  
count=$((count + 1))
```

10.6 Arithmetic Command

- The command `(( ... ))` evaluates integer arithmetic as a command.

Increment counter

```
count=0  
((count++))
```

- It is often used for increments and numeric tests.

Numeric test

```
count=3  
((count > 0)) && echo "yes"
```

10.7 let - Arithmetic Builtin

- The let builtin evaluates arithmetic expressions.

Add with let

```
count=1  
let "count=count+1"
```

- It is older and less visually clear than `$(( ))` or `(( ))`.

Print result

```
echo "$count"
```

11

Section 11

Substitution and Special Variables

11.1 Command Substitution

- Command substitution runs a command and inserts its output.

Capture date

```
today=$(date +%Y-%m-%d)
```

- The modern form uses dollar sign and parentheses.

Print captured value

```
echo "$today"
```

11.2 Backticks Versus Dollar Parentheses

- Backticks are the older form of command substitution.

Old form

```
today=`date +%Y-%m-%d`
```

- The dollar-parentheses form is easier to nest and read.

Modern form

```
today=$(date +%Y-%m-%d)
```

11.3 Command Substitution Newlines

- Command substitution removes trailing newline characters from command output.

Capture output

```
files=$(printf "a\nb\n")
```

- Internal newlines can remain inside the captured value.

Show captured text

```
printf "[%s]\n" "$files"
```

11.4 Process Substitution Intro

- Process substitution lets a command read the output of another command as if it were a file.

Compare two streams

```
diff <(sort a.txt) <(sort b.txt)
```

- Bash commonly uses the forms <(command) and >(command).

Read generated input

```
while read -r x; do echo "$x"; done < <(printf "a\n")
```

11.5 \$? - Last Exit Status

- \$? expands to the exit status of the most recent foreground command.

Show status

```
false  
echo "$?"
```

- Zero usually means success, and nonzero usually means failure.

Check after command

```
ls missing.txt  
echo "$?"
```

11.6 \$\$ - Current Shell PID

- \$\$ expands to the process ID of the current shell.

Print shell PID

```
echo "$$"
```

- A process ID is a number the operating system uses to identify a running process.

Use in a name

```
tmp="run-$$log"  
echo "$tmp"
```

11.7 \$! and \$_

- \$! expands to the process ID of the most recent background command.

Capture background PID

```
sleep 5 &  
echo "$!"
```

- \$_ often expands to the last argument of the previous simple command.

Show last argument

```
mkdir -p logs  
echo "$_"
```

1 2

Section 12

Formatting, Comparisons, and Review

12.1 printf Formatting

- Printf uses placeholders to control how values are displayed.

Print two strings

```
printf "Name: %s\n" "$name"
```

- The %s placeholder prints a string value.

Print table row

```
printf "%-10s %s\n" "$user" "$role"
```

12.2 printf Versus echo -e

- Echo behavior for escape sequences can vary between shells and systems.

Portable newline

```
printf "one\ntwo\n"
```

- Printf handles escapes through its format string in a predictable way.

Avoid relying on echo -e

```
echo -e "one\ntwo"
```

12.3 Boolean Strings

- Bash does not have a separate Boolean variable type for ordinary strings.

Store a flag

```
enabled=true
```

- Scripts often use values such as true, false, yes, no, 1, or 0.

Check a flag

```
[ "$enabled" = true ] && echo "on"
```

12.4 String Compare Preview

- String comparisons test text values.

Compare strings

```
[ "$mode" = "dev" ] && echo "dev mode"
```

- The single equal sign is commonly used inside the test command for string equality.

Compare nonempty

```
[ -n "$name" ] && echo "$name"
```

12.5 Numeric Compare Preview

- Numeric comparisons treat values as numbers instead of text.

Compare integers

```
count=3
((count > 1)) && echo "many"
```

- Arithmetic commands are a clear Bash-style way to compare integers.

Add before compare

```
total=$((a + b))
((total >= 10))
```

12.6 Common Quoting Bugs

- Missing quotes can split file names that contain spaces.

Buggy path use

```
file="my notes.txt"  
cat $file
```

- Missing quotes can expand wildcard characters from data into filenames.

Safer path use

```
file="my notes.txt"  
cat "$file"
```

12.7 Checkpoint - Variables and Quoting

- You should be able to assign variables without spaces and read them with expansion.

Safe variable use

```
name="Ada Lovelace"  
printf "%s\n" "$name"
```

- You should know when double quotes, single quotes, and braces change behavior.

Safe argument loop

```
for arg in "$@"; do  
    printf "%s\n" "$arg"  
done
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

02 · Variables

